A Conditional Proof Architecture for the Two-Player $3n \pm 1$ Game

Grisha Pochuev
n_854@mail.ru
https://github.com/Grisha-Pochuev/3n-plus-minus-1-game
doi:10.5281/zenodo.21844684

Hostile-audit revision, 12 August 2026

Abstract

We consider the impartial two-player game on positive odd integers in which a move replaces $n > 1$ by the odd part of either $3n - 1$ or $3n + 1$, and the player who reaches 1 wins. Arbitrary play need not terminate, so the relevant question is whether a draw can survive optimal play. We give a binary normal form with one expanding branch and one strictly contracting branch. The global proposal marks finite outcome proofs by their canonical heights and ranks finite multisets of these tokens by an ordinal. A hostile audit finds that the proposed fixed-fibre normalizer is not yet proved: a long high-return step compares a locally constructed pair without establishing that the pair is already carried by the incoming rank, and an unbounded $D = 2$ loss-anchored module lacks a complete same-token exit proof. Consequently this manuscript does not claim that all draw positions have been excluded. It gives a rigorous reduction, proves the sound abstract criterion that would finish the problem, and states the two remaining obligations precisely. A detailed proof supplement, regression tests, and the exact status of the ongoing Lean formalization are included in the accompanying repository. The repository also contains a declarative global-routing certificate, a small independent checker, and a Lean proof of the general well-founded-certificate metatheorem. Their exact trust boundary is stated explicitly: the global assembly is machine-checked conditional on four local refinement obligations that remain part of the conditional argument.

1 Introduction

The game studied here appears as Conway's *Beans-Don't-Talk* game in Guy's accounts [3, 4] and more recently as the $3n+-1$ game of Althöfer, Hartisch, and Zipproth [2]. Its similarity to Collatz iteration is superficial in one important respect: the two signs are strategic choices by alternating players.

The specific problem addressed here is the two-player problem stated on Althöfer's Collatz-problems page: prove that every odd starting value reaches 1 when both players act optimally [1]. The formulation on that page and the game rules used below agree exactly.

For the player to move, a position is WIN if some move leads to a LOSS position, and is LOSS if every move leads to a WIN position. Because the game graph is infinite, these two inductive classes need not exhaust all positions. We call every remaining position DRAW.

Thus finite-game backward induction cannot be used without a separate well-foundedness argument.

Indeed arbitrary play may cycle; for example $5 \to 7 \to 5$ is possible. The target statement concerns optimal play and is therefore strictly stronger than the existence of some terminating choice and different from termination under all choices.

Current status: OPEN. The long high-return token-provenance edge and the unbounded $D = 2$ loss-anchored routing module have not passed hostile audit.

Proposition 1.1 (Target conjecture)**.** *For every positive odd starting integer, optimal play reaches* 1 *after finitely many moves. Equivalently, the game has no* DRAW *positions.*

Independent verification package. The proof supplement and the independently runnable verification artifacts are maintained at

`https://github.com/Grisha-Pochuev/3n-plus-minus-1-game`.

An external reader can clone the repository and run `python audit.py` to check the documented arithmetic regressions, finite proof certificates, and the conditional global-routing assembly. The same repository contains the negative tests and Lean-checked certificate metatheory, together with an explicit account of what remains human-verified.

This manuscript presents the proposed dependency chain. Detailed local case arguments and their current statuses are contained in the numbered proof supplement [5]; section numbers below refer to that supplement. The supplement, executable checks, certificate inventory, and formalization sources are openly available at `https://github.com/Grisha-Pochuev/3n-plus-minus-1-game`.

2 Rules and binary normal form

For a positive integer x, write $\mathrm{odd}(x)$ for the result of dividing x by the largest power of two that divides it. From an odd $n > 1$ the two children are

$$\mathrm{odd}(3n-1), \qquad \mathrm{odd}(3n+1).$$

The state 1 is terminal and losing for the player to move.

Write $n = 2m + 1$ and define

$$F(m) = \left\lceil \frac{3m}{2} \right\rceil.$$

Let $R(m)$ be the integer obtained by deleting the maximal alternating suffix of the binary expansion of m. For instance, $R(1001_2) = 10_2$ and $R(10101_2) = 0$.

Proposition 2.1 (First normal form)**.** *For $m > 0$, the two children in m-coordinates are exactly*

$$F(m), \qquad F(R(m)).$$

Proof. We have $3n - 1 = 2(3m + 1)$ and $3n + 1 = 2(3m + 2)$. Exactly one of $3m + 1, 3m + 2$ is odd. Splitting on the final bit of m gives $F(m)$ from that expression. In the other expression, each additional division by two crosses one alternating bit of m; the first repeated adjacent bit stops the division. This is precisely deletion of the maximal alternating suffix. The four prefix/final-bit cases are given in the supplement's normal-form proof. □

Every child in Proposition 2.1 lies in the image of F. Representing $F(q)$ by q gives the conjugated game

$$A(q) = F(q) = \left\lceil \frac{3q}{2} \right\rceil, \qquad B(q) = R(F(q)),$$

with terminal state $q = 0$.

Proposition 2.2 (Contracting branch)**.** *For every $q > 0$, $B(q) < q$.*

Proof. Deleting a nonempty binary suffix gives

$$B(q) \le \left\lfloor \frac{F(q)}{2} \right\rfloor \le \left\lfloor \frac{3q+1}{4} \right\rfloor < q$$

for $q \ge 2$, while $B(1) = 0$ directly. □

The difficulty is that A expands and the length of the suffix deleted by R is unbounded. In particular no fixed residue modulus determines R.

3 Finite proof tokens

The ordinary inductive outcome construction gives every certified WIN or LOSS position a finite canonical proof height. Terminal state 0 has height zero. A WIN certificate selects a lower-height LOSS child; a LOSS certificate contains both lower-height WIN children.

During the draw analysis we retain the exact certified endpoints used by these finite proofs. A marked configuration therefore carries a finite multiset M of proof heights. These are *tokens*: their identities and source provenance are retained, not merely their largest height or their cardinality.

Lemma 3.1 (Token multiset rank; Supplement Section 129)**.** *For a finite multiset M of proof heights define*

$$\Phi(M) = \#_{h \in M} \omega^h,$$

where $\#$ denotes natural ordinal sum. Replacing one token of height h by finitely many certified descendants of heights strictly below h strictly decreases Φ.

The use of a multiset, rather than a scalar maximum, is essential: a routing step may split one obligation into several lower ones. Equality of Φ alone is not used to identify configurations. The transition inventory proves that every token-changing edge is strict, hence an equal-Φ route preserves the actual token multiset.

4 Marked factor routing

Long constant binary tails admit coordinates

$$Q_D^g(C) = C2^D - g, \qquad g \in \{0, 1\},$$

with C odd. The minimum-source analysis produces marked factor scans in these coordinates. Their role is to keep a finite outcome witness attached while the arithmetic route is normalized.

Lemma 4.1 (Factor attachment; Supplement Sections 130–135)**.** *Every marked factor scan retains its incoming finite proof tokens. For a marked source $b = Q_D^q(C)$ with $D \geq 3$, let y be its marked* LOSS *child and q its marked* WIN *child. There is a common* WIN *child r with*

$$h(r) \leq h(y) - 1.$$

Before an inner branch is followed, the scan replaces the carried occurrence of y by r. Every exit retains this lower token and its exact source attachment. The remaining rows $D = 1, 2$ are exact lifts over an already marked token and create no unranked fresh witness.

This statement closes a common logical gap: reaching a later finite endpoint is insufficient unless it is comparable with the witness already used in the outer rank. Sections 130–135 prove precisely that comparison.

5 Well-founded fibres

We use the following standard projection principle in a form adapted to the marked relation.

Lemma 5.1 (Well-founded fibre lemma; Supplement Section 132)**.** *Let a transition relation carry a projection into a well-founded order. Suppose the projection never increases, every projection-changing edge is strictly decreasing, and the transition relation restricted to each fixed projection fibre is well-founded. Then the full transition relation is well-founded.*

The proof is well-founded induction on the projection, followed by induction inside the current fibre. The formulation permits inner finite heights that are incomparable between different routes; only the retained outer projection must be monotone.

Proposition 5.2 (Proposed fixed-fibre normalizer; open)**.** *Fix the retained coefficient source and the outer proof-token multiset. The complete generic obligation/factor routing relation is well-founded.*

Remark 5.3 (Failure point exposed by hostile audit)**.** *Canonical A-streaks have a four-side-test exit. Valuation-two B transfers install and then lower a finite local witness. Higher valuations either lower the source or enter a factor fork that pays in source or proof height. A high return re-enters only the marked factor rows of the preceding section.*

Within the fibre, the internal source/token measure has the form

$$\omega^{\omega} a + \Psi(N),$$

where a is the inner source and N its finite token multiset. The proposed long high-return edge replaces a pair u, c by lower local descendants, but the incoming macro-configuration has not been shown to carry the exact occurrences u, c. Therefore the displayed local height inequalities do not yet decrease $\Psi(N)$. The short $D = 2$ return also needs a complete unbounded same-token rank or exit certificate. Until both statements are proved, the proposition is open and cannot be used in a no-draw proof.

The two defects may be one lifecycle problem. In the $D = 2$ row the direct DRAW lift has the exact form

$$w = Q_m^{\delta}(J(y)), \qquad m \geq 2, \qquad y \text{ a retained LOSS token.}$$

Its phase is B-selecting. If $B(w)$ is DRAW, then $B(w) < w$ is a strict numerical exit. Otherwise $B(w)$ is WIN and $w \to B(w)$ is a concrete DRAW-to-WIN boundary while the continuing DRAW is $A(w)$. Thus this row does expose a legitimate finite witness. What is still missing is a proof that this exact boundary witness is carried through every zero-rank routing edge until the next high return, where it is replaced by a certified descendant or an earlier source coordinate has already decreased.

6 Entry completeness

Two analyses feed the normalizer. Sections 14–27 treat a least coefficient source through constant-tail frames. Sections 69–90 treat height one separately; its resonance is coupled across consecutive factor levels and is not replaced by a fixed-depth computation.

Lemma 6.1 (Entry trichotomy; conditional target of Supplement Section 137)**.** *Every outcome-compatible continuation from the minimum-source and height-one analyses has one of three outputs:*

1. *strict coefficient-source descent;*
2. *strict descent of the retained proof-token multiset rank;*
3. *entry into the fixed-fibre normalizer of Section 136.*

There is no fourth arithmetic exit.

The supplement audits the earlier case splits against this trichotomy. This is the exhaustiveness step that connects the local arithmetic to the global well-founded relation.

7 Conditional exclusion of draws

Theorem 7.1 (Conditional no-draw criterion)**.** *If the proposed fixed-fibre normalizer is proved with exact token identities, including the complete $D = 2$ module, and every entry in the trichotomy is a finite outcome-compatible refinement, then the game has no* DRAW *positions.*

Proof. Assume a DRAW exists and choose one with least coefficient source s. Following the exhaustive outcome diamonds produces a marked macro-configuration that still contains an actual DRAW. By the entry trichotomy, every macrostep either reaches a DRAW below s, strictly decreases the retained token rank Φ, or stays within a fixed normalizer fibre.

The first alternative contradicts minimality of s. The second cannot occur infinitely because ordinals are well-founded. The third cannot occur infinitely by the fixed-fibre normalizer lemma. The well-founded fibre lemma therefore shows that the complete marked transition relation admits no infinite path.

On the other hand, a DRAW position has no LOSS child and has at least one DRAW child. Repeatedly following such a child through the finite macro-normalizations would give an infinite marked transition path, a contradiction. Thus no DRAW exists.

Every state is consequently WIN or LOSS with finite canonical proof height. Optimal play from a WIN state selects a lower-height LOSS child, and every move from a LOSS state reaches a lower-height WIN child. The height decreases until conjugated state 0, corresponding to original state 1.

Finally, if an original odd state is divisible by three, then after either legal move the odd part of $3n \pm 1$ is not divisible by three and lies in the conjugated image. Hence the conclusion covers every odd starting state under the stated hypotheses. □

8 Verification status and reproducibility

The repository distinguishes four kinds of evidence.

First, the mathematical supplement labels each claim as proved, computationally verified, conjectural, disproved, or an unverified lead. Second, standard-library Python code checks the exact normal form, descent identities, routing arithmetic, and bounded retrograde soundness. These finite runs are regression evidence and are not used as a proof of the infinite theorem.

Third, the repository contains a finite declarative inventory of the global routing assembly. It has 15 control states, 46 transition schemas, and 12 guard partitions, including symbolic unbounded valuation and tail-length intervals. An independent standard-library Python checker verifies source integrity, complete coverage of the *declared* cases, one rule per case, lexicographic reset discipline, and acyclicity of the 17 equal-rank control edges. The remaining 29 schemas have an explicit strict rank component.

The checker prints the status `CONDITIONAL_MACHINE_CHECK`. Four obligations deliberately remain on the human side of the trust boundary:

1. the universal arithmetic and coordinate identities attached to each macro schema;
2. refinement of the declared semantic guards to every legal game case;
3. outcome compatibility of the selected DRAW continuation and the separately carried finite tokens;
4. finite productivity of every macro-normalization.

Thus acceptance of the JSON certificate validates only the declared abstract size-change graph. It does not validate the unresolved long high-return token installation, the direct-DRAW $D = 2$ lift, or a complete refinement from the original move relation. The certificate is therefore a conditional design artifact rather than evidence that the target conjecture has been proved.

Fourth, a pinned Lean 4 project proves the exact unbounded alternating-suffix normal form and its conjugacy with the original signed game. It formalizes finite WIN/LOSS trees, the DRAW-successor lemma, the one-to-at-most-two proof-token rank used by the inventory, the four-component macro rank, and the 17-edge equal-rank control DAG. The arithmetic front additionally checks the constant-tail and unique source coordinates, the minimum-source boundary phase and reverse factor-three frame, the two lifted-return source identities, and the phase rows through Supplement Section 19. It then proves that a complete semantic macro refinement would exclude every DRAW state. The valuation-two, valuation-three, and arbitrary higher cases of the Section 20 large diamond, including its outcome contradiction, are also kernel-checked. The Lean sources use no problem-specific axioms and contain no `sorry` or `admit`. The minimum-source outcome fingerprint, returned-source drop, and exclusion of all four phase-mismatch classes in Section 20 are kernel-checked as well. Section 21 is now carried through the exact adjacent pair, the strict common-child source bound, and the exclusion of all four long-suffix phase-match classes modulo 128. For the four surviving suffix-length-three rows, Section 22 is also kernel-checked: the common child is reached through both branches of the returned WIN

state and every concrete winning proof tree drops by at least two levels. Section 23 now checks the exact lifted frames, returned-child orientation, strict source drop, and forced outcome fork in all eight suffix-length-two rows. Section 24 checks the universal A-selecting side-child identity and its two-level proof-height drop. Section 25 checks the universal B-selecting large diamond and transfers both permitted DRAW entries to the adjacent frame over the selected source. The main Section 26 filter is also checked: source survival, exact valuation two, and the eight residue classes modulo 256 are proved equivalent; the next phase is then split exhaustively into the stated sixteen classes modulo 512. Section 27 checks preservation of the exact DRAW obligation under valuation-two B transfers, strict source descent for every higher valuation, and exclusion of three further surviving B transfers. Lean does not yet construct the complete concrete refinement: Section 28 now also checks the strict side-source drop and the exhaustive split between the canonical A-continuation and three fully marked bounded factor frames. The Section 29 opposite-tail switch and later universal guard coverage, outcome-compatible macro selection, and finite productivity remain the explicit boundary recorded in the coverage table.

This separation is part of the result: it permits an auditor to reproduce the computations without confusing them with the well-foundedness proof, and to measure future formalization progress without importing the desired theorem as an axiom.

From a clean checkout, the complete locally machine-checkable audit is:

```
python audit.py
```

It runs the complete unit and regression test suite, checks the documented finite arithmetic range, validates the global assembly certificate, generates a finite outcome proof DAG, and validates that DAG with a separate checker. The global certificate alone is checked by

```
python scripts/verify_global_certificate.py
```

The Lean metatheory is checked independently by `lake build` in the `formal` directory. Exact commands, expected output, negative tests, and the human audit route are recorded in the repository's `START_HERE.md` and `AUDIT.md` files.

9 Known proof hazards

The proposed argument does not propagate a least draw indefinitely along the expanding branch, infer a theorem from a search depth, use a fixed modulus for an unbounded suffix, or infer winning play from a merely terminating one-player strategy. The hostile audit nevertheless found that its long high-return edge does not yet establish that the locally compared witnesses were retained by the incoming configuration. Separate termination of the obligation and factor subsystems would not by itself imply termination of their union.

10 Conclusion

The binary normal form isolates the contracting branch but does not itself rule out strategic cycles. Marked finite outcome witnesses provide a plausible global control, but the current normalizer has not yet proved the lifecycle of the long high-return pair or the complete unbounded $D = 2$ routing module. Therefore finite optimal play from all odd starts remains open.

Data, code, and correspondence

The proof supplement, source code, tests, certificate files, Lean sources, and reproducibility records are available at

`https://github.com/Grisha-Pochuev/3n-plus-minus-1-game`.

The repository is the authoritative location for the evolving verification artifacts. An earlier archived version and source are available at

doi:10.5281/zenodo.21844684.

Questions, counterexamples, and audit reports may be sent to n_854@mail.ru.

References

[1] Ingo Althöfer. Collatz Problems: The Two-Player $3n+-1$ Game. `https://althofer.de/collatz-prizes.html`, 2026. Problem statement; accessed 8 August 2026.

[2] Ingo Althöfer, Michael Hartisch, and Thomas Zipproth. Analysis of a collatz game and other variants of the $3n+1$ problem. In *Advances in Computer Games*, pages 123–132, 2024.

[3] Richard K. Guy. John isbell's game of beanstalk and john conway's game of beans-don't-talk. *Mathematics Magazine*, 59(5):259–269, 1986.

[4] Richard K. Guy. Unsolved Problems in Combinatorial Games. In *Games of No Chance*, volume 29 of *MSRI Publications*. Cambridge University Press, 1996. Problem 42.

[5] Grisha Pochuev. Optimal $3n \pm 1$ Game: Proof and Verification Repository. `https://github.com/Grisha-Pochuev/3n-plus-minus-1-game`, 2026. Proof supplement, source code, symbolic certificates, tests, reproducibility records, and Lean formalization status; published preprint archived at `https://doi.org/10.5281/zenodo.21844684`.